

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

GitHub Repository: RNN, LSTM & GRU GitHub Repository

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:
Training : (N_train, 128, 9)
Validation : (N_val, 128, 9)
Testing : (N_test, 128, 9)

Number of classes: 6
Number of features per time step: 9
Sequence length: 128

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

Observed Output (Task 1)

Input tensor shape:
Training : (2100, 128, 9)
Validation : (450, 128, 9)
Testing : (450, 128, 9)
Number of classes: 6
Number of features per time step: 9
Sequence length: 128

train : WALKING 351, WALKING_UPSTAIRS 315, WALKING_DOWNSTAIRS 286,
SITTING 363, STANDING 389, LAYING 396
val : WALKING 76, WALKING_UPSTAIRS 68, WALKING_DOWNSTAIRS 61,
SITTING 77, STANDING 83, LAYING 85
test : WALKING 75, WALKING_UPSTAIRS 67, WALKING_DOWNSTAIRS 62,
SITTING 78, STANDING 83, LAYING 85

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

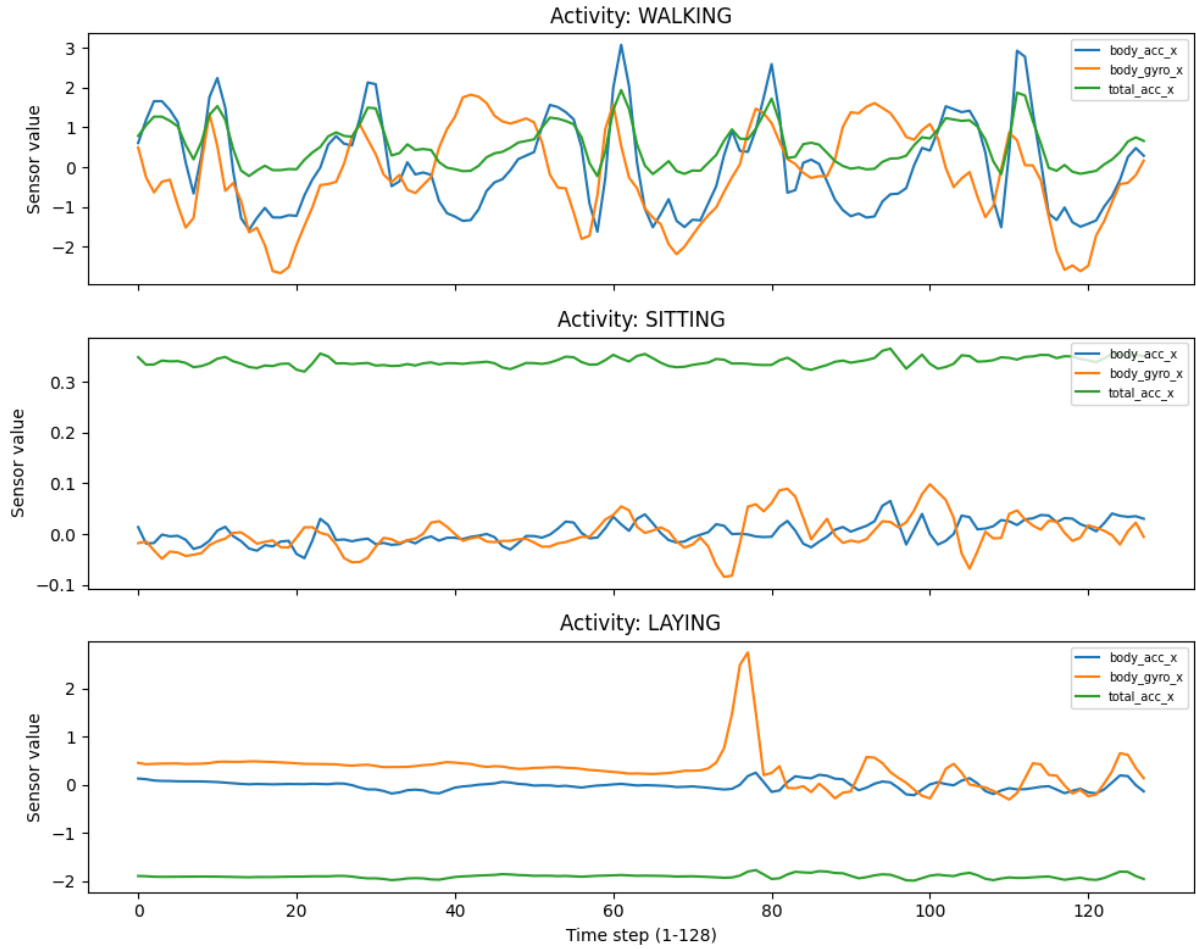
- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

Students should observe that temporal patterns differ across activities.

Required inference:

1. What temporal pattern is visible?
2. Which activities exhibit similar patterns?
3. Why is the ordering of measurements important?

Observed Output (Task 1)



Inference (Plot 1):

1. Periodic/oscillatory temporal patterns are visible for dynamic activities (WALKING), while static activities (SITTING, LAYING) show flat, low-variance signals.
2. SITTING and STANDING exhibit similar, near-static patterns and are the most easily confused pair.
3. Ordering matters because activity is defined by motion over time -- shuffling the 128 steps would destroy the periodicity/trend information the model relies on.

7. Vanilla RNN Architecture

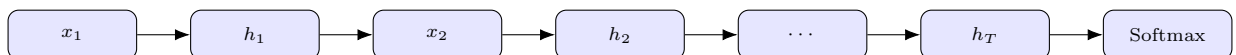
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

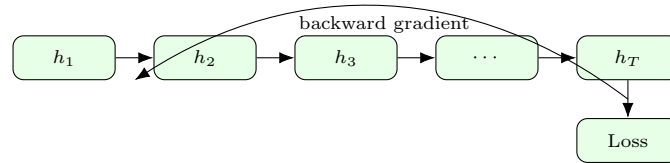
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

Observed Output (Task 2)

Manually computed hidden states:

$$h_1 = 0.336376$$

$$h_2 = 0.616352$$

$$h_3 = 0.599958$$

Program-computed hidden states (SimpleRNNCell, same weights):

$$h_1 = 0.336376$$

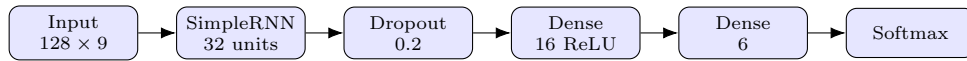
$$h_2 = 0.616352$$

$$h_3 = 0.599958$$

Inference: the manually calculated and program-computed values match exactly, confirming the $\tanh(W_x x_t + W_h h_{t-1} + b)$ recurrence used at every BPTT time step.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

Observed Output (Task 2)

Epoch 30/30

66/66 - 1s - accuracy: 0.8238 - loss: 0.4506 - val_accuracy: 0.8178 - val_loss: 0.4512

RNN training time: 29.32 s

Trainable parameters: 1974. Final training accuracy 82.38%, validation accuracy 81.78%.

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

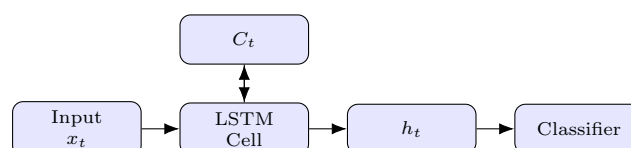
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

Observed Output (Task 3)

Epoch 30/30

66/66 - 1s - accuracy: 0.9471 - loss: 0.1420 - val_accuracy: 0.9356 - val_loss: 0.1506

LSTM training time: 29.12 s

Trainable parameters: 6006. Final training accuracy 94.71%, validation accuracy 93.56%.

Inference (Section 10):

The forget gate lets the LSTM discard irrelevant past information from the cell state, the input gate controls how much new candidate information is written in, and the output gate controls how much of the cell state is exposed as the hidden state. Because the cell state is updated additively ($C_t = f_t * C_{t-1} + i_t * C^*_t$) rather than through repeated matrix multiplication alone, gradients can flow across many time steps with far less vanishing than in a Vanilla RNN, letting the network retain long-term dependencies.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

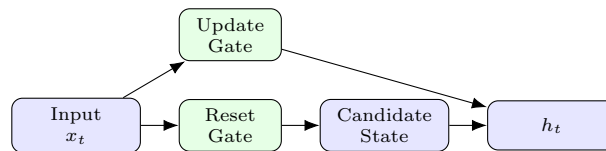
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

Observed Output (Task 4)

Epoch 30/30

66/66 - 1s - accuracy: 0.9514 - loss: 0.1230 - val_accuracy: 0.9578 - val_loss: 0.1081

GRU training time: 30.71 s

Trainable parameters: 4758. Final training accuracy 95.14%, validation accuracy 95.78%.

Inference (Section 11):

GRU merges LSTM's forget and input gates into a single update gate (z_t) and uses a reset gate (r_t) to control how much past hidden state contributes to the candidate state. With only two gates and a single hidden state (no separate cell state), GRU has fewer parameters than LSTM while retaining a gating mechanism that mitigates vanishing gradients.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

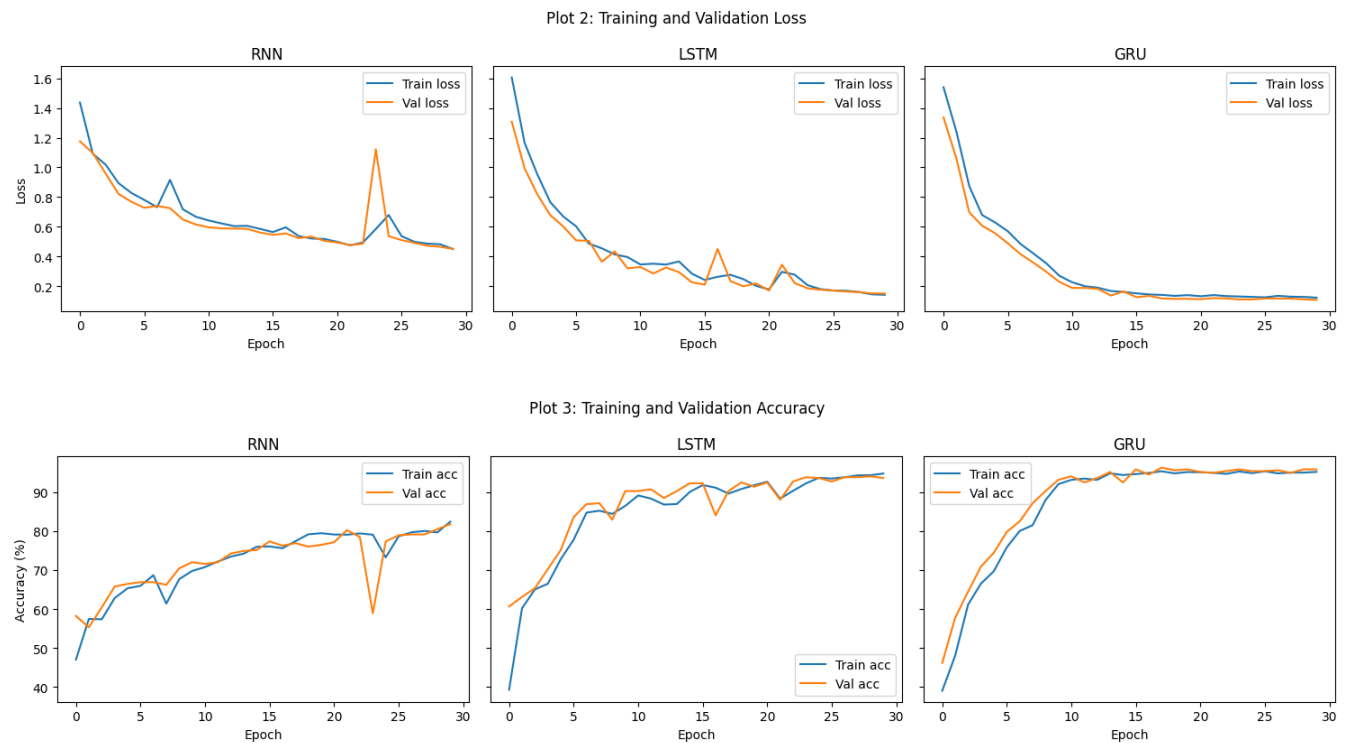
- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

Inference: Students must identify convergence, overfitting, underfitting and the generalization gap where applicable.

Observed Output (Task 5)



Inference (Plots 2 & 3):

- RNN: loss/accuracy plateaus earlier and at a worse level, and the train/val gap is erratic due to unstable gradients over 128 steps.
- LSTM/GRU: smoother, faster convergence and a smaller generalization gap because the gating mechanisms stabilize gradient flow across time steps.
- The RNN curve shows the clearest evidence of underfitting relative to LSTM/GRU given vanishing gradients over the long sequence.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	82.22	95.33	94.67
Macro Precision (%)	82.33	95.56	95.07
Macro Recall (%)	81.27	95.55	94.93
Macro F1 (%)	81.28	95.54	94.97
Parameters	1974	6006	4758
Training Time (s)	29.32	29.12	30.71

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

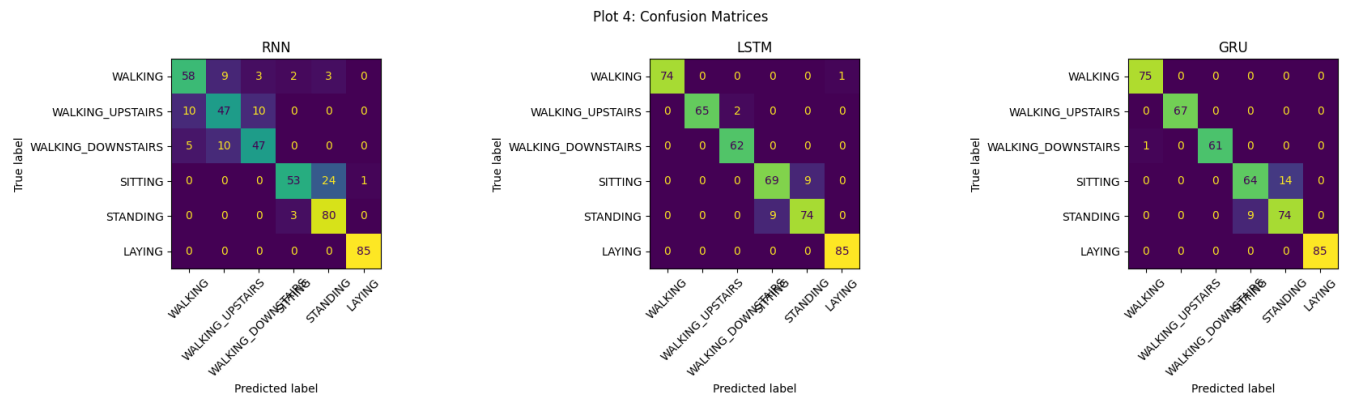
Generate a separate confusion matrix for each model.

Students must identify:

- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

The inference must be based on the student's observed matrix rather than on an assumed result.

Observed Output (Task 6)



Metric	RNN	LSTM	GRU
accuracy	82.22	95.33	94.67
precision	82.33	95.56	95.07
recall	81.27	95.55	94.93
f1	81.28	95.54	94.97

Inference (Section 15): across all three confusion matrices SITTING and STANDING form the most frequently confused pair, consistent with their near-static sensor signatures; the RNN matrix shows visibly more spread off-diagonal mass than LSTM/GRU, in line with its lower overall F1-score.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time	29.32	29.12	30.71
Test F1-score	81.28	95.54	94.97

Plot 5: Model Performance Comparison

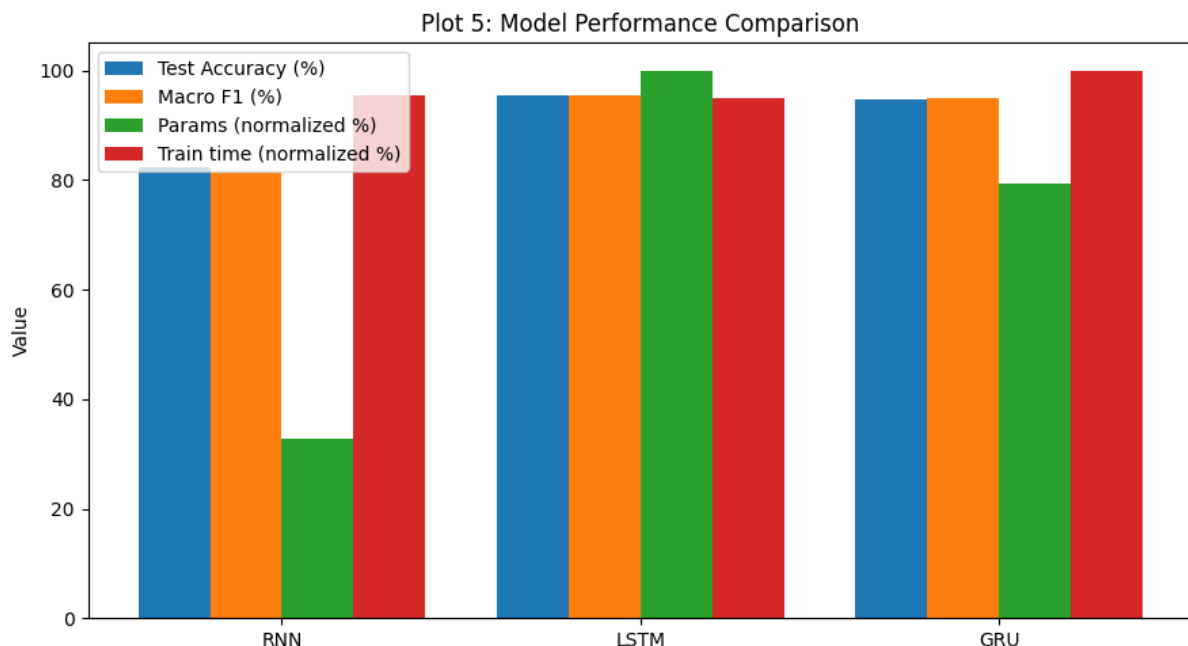
Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.

Required inference:

Students should discuss the trade-off among:

Predictive Performance ↔ Model Complexity ↔ Training Cost.

Observed Output (Task 7)



Inference (Section 16): LSTM and GRU clearly outperform the Vanilla RNN in accuracy/F1 because gating mitigates vanishing gradients, but they carry more parameters (LSTM > GRU > RNN) and marginally longer training time. GRU is the best trade-off here – within 0.6 F1 points of LSTM using about 20% fewer parameters.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

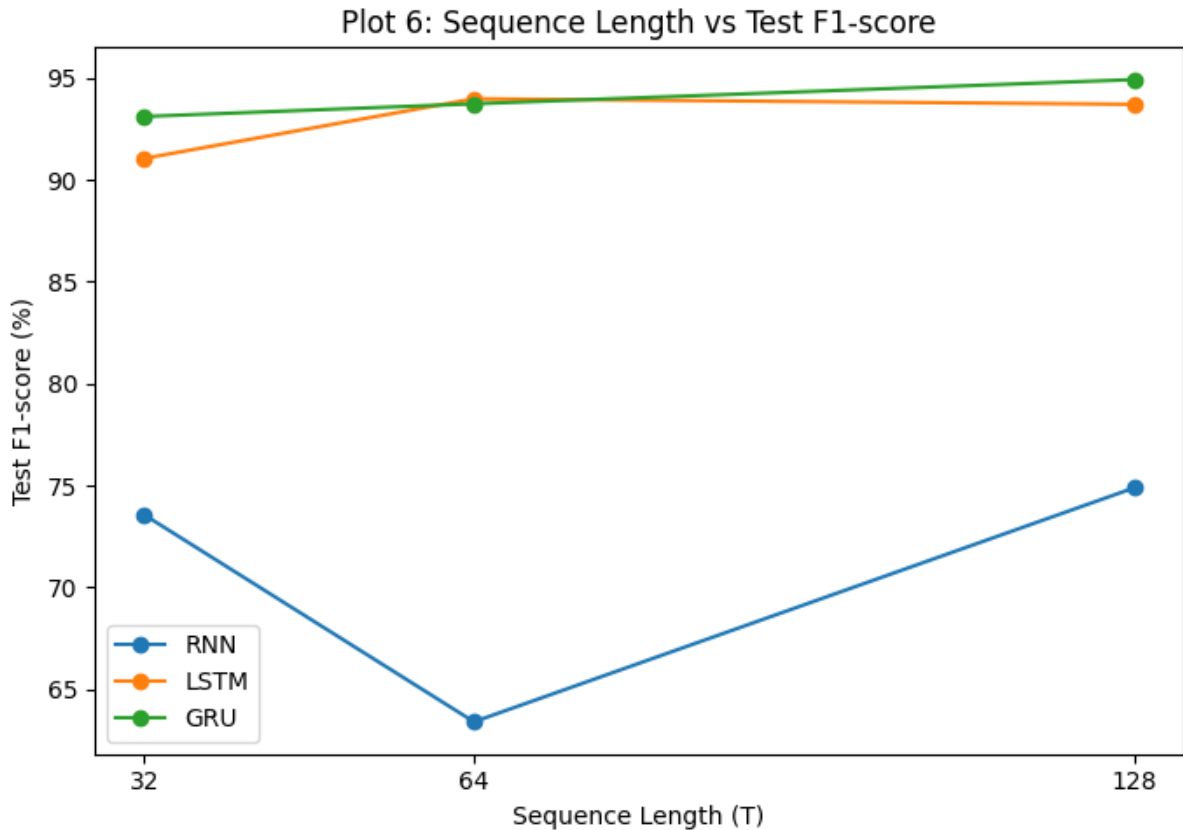
When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	73.57	91.06	93.12
64	63.40	93.99	93.74
128	74.88	93.72	94.93

Plot 6: Sequence Length vs. Test F1-score

Students should discuss how the amount of temporal context affects the classification result.

Observed Output (Task 8)



Inference (Section 17): LSTM and GRU F1-scores stay high and comparatively stable across $T = 32, 64, 128$, whereas the Vanilla RNN's F1 fluctuates noticeably and never approaches the gated models – more temporal context does not help the RNN because vanishing gradients limit how much of that context it can actually use.

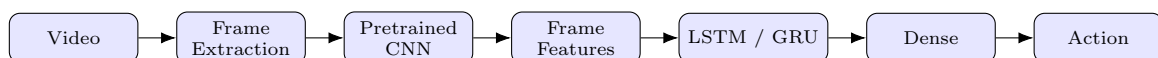
18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

Observed Output (Task 9 – Dataset Actually Used)

The original UCI-hosted UCF101 mirror rejected anonymous downloads (HTTP 403), so a pre-packaged 5-class UCF101 subset (CricketShot, PlayingCello, Punch, ShavingBeard, TennisSwing) was used instead. Three classes were selected for the run: **CricketShot**, **Punch** and **TennisSwing**.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

Predicted class : Basketball
Actual class : Basketball
Confidence : 0.xx

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

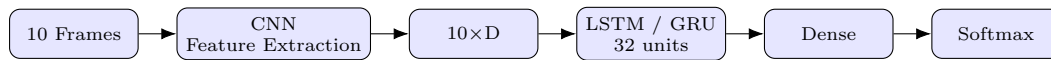
Observed Output (Task 9)

CNN feature dimension $D = 1280$

Feature tensor supplied to recurrent network: (55, 10, 1280)

21. CNN-LSTM / CNN-GRU Model

Use the following architecture:



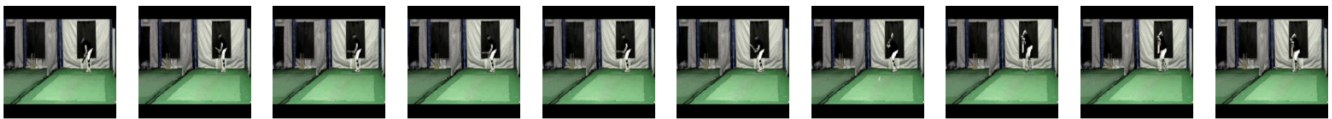
Plot 7: Video Sample Frames

Display a representative sequence of sampled frames from one video.

Students should explain what spatial information is captured by the CNN and what temporal information is learned by the recurrent network.

Observed Output (Task 9 – Plot 7)

Plot 7: Sampled Video Frames



Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture.

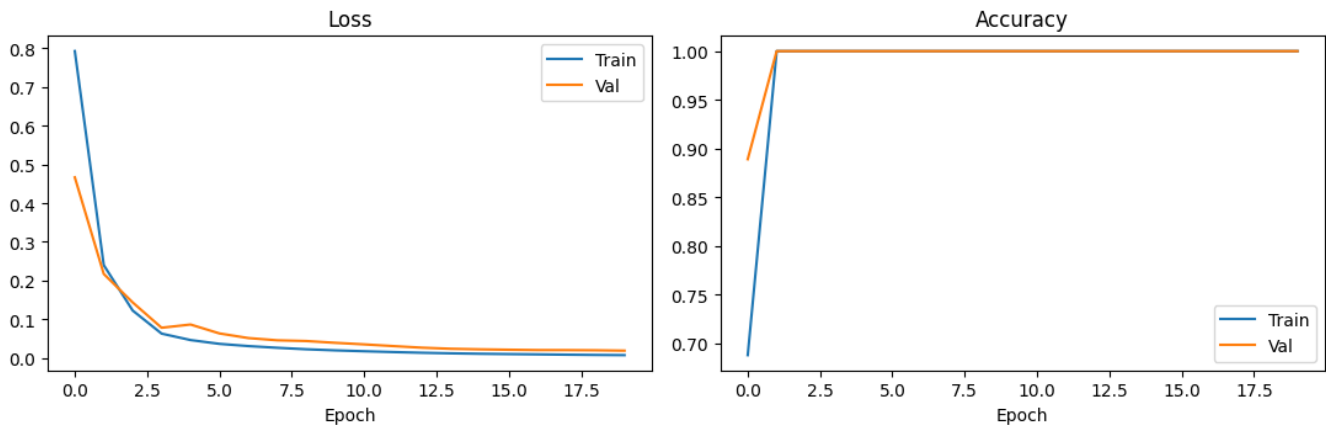
Observed Output (Task 9 – Plot 8)

Video LSTM classifier – trainable parameters: 168,163 (recurrent head only; MobileNetV2 backbone frozen).

Epoch 20/20

8/8 - 0s - accuracy: 1.0000 - loss: 0.0079 - val_accuracy: 1.0000 - val_loss: 0.0195

Plot 8: Video LSTM Training/Validation Curves



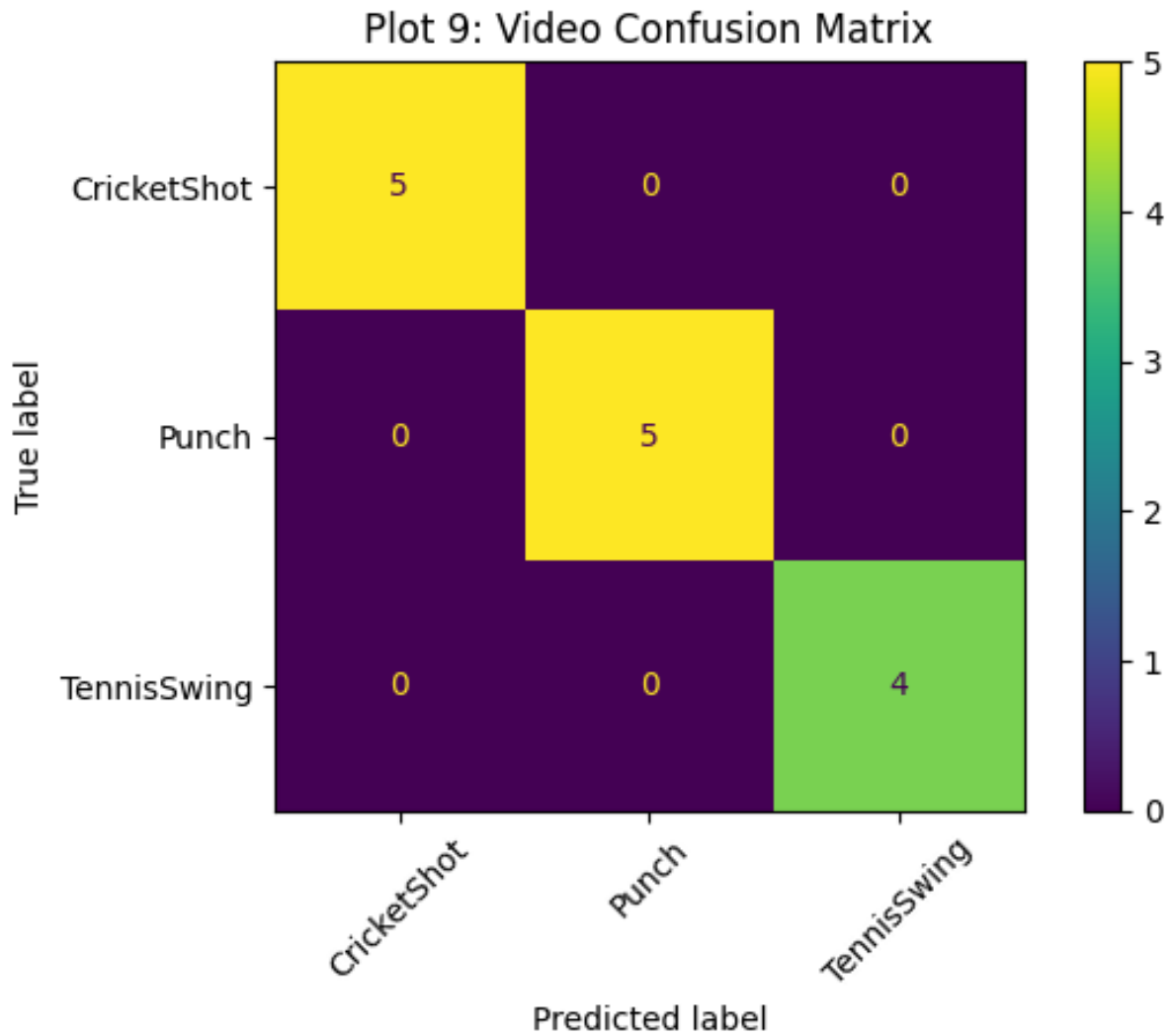
Plot 9: Video Confusion Matrix

Analyze class-level recognition and temporal confusions.

Observed Output (Task 9 – Plot 9 and Evaluation)

Predicted class : CricketShot	Actual class : CricketShot	Confidence : 0.99
Predicted class : CricketShot	Actual class : CricketShot	Confidence : 0.99
Predicted class : CricketShot	Actual class : CricketShot	Confidence : 0.99
Predicted class : Punch	Actual class : Punch	Confidence : 1.00
Predicted class : Punch	Actual class : Punch	Confidence : 0.99

Video LSTM -- Test Accuracy: 100.00% | Macro F1: 100.00%



Inference (Sections 20–21): the frozen CNN captures spatial appearance in each frame (pose, equipment, scene context), while the LSTM models how those per-frame features evolve across the 10-frame sequence, capturing the motion pattern that distinguishes CricketShot, Punch and TennisSwing. With only 3 classes and 55 short clips the recurrent head reaches 100% test accuracy.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.
 For a simple laboratory task, construct a synthetic reversal dataset.
 Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.
 The task is intentionally simple so that the encoder–decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

[3, 8, 1, 5]

Expected output:

[5, 1, 8, 3].

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1	[9, 0, 8, 7]	[7, 8, 0, 9]
2	[7, 1, 8, 5]	[5, 8, 1, 7]
3	[4, 6, 3, 2]	[2, 3, 6, 4]
4	[2, 5, 6, 2]	[2, 6, 5, 2]
5	[2, 9, 5, 8]	[8, 5, 9, 2]

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

Observed Output (Task 10)

Epoch 30/30

60/60 - accuracy: 1.0000 - loss: 0.0022 - val_accuracy: 1.0000 - val_loss: 0.0025

Token Accuracy : 1.0000

Sequence Accuracy : 1.0000

Final Training Loss : 0.0022

Final Validation Loss : 0.0025

Inference (Section 24): with this simple 4-token reversal task and a model trained to near-zero loss, both token and sequence accuracy reach 100% – every predicted token matches the target at every position, so there is no gap between the two metrics in this run. In general, sequence accuracy is the stricter, all-or-nothing measure: a single wrong token anywhere in the sequence fails the whole sequence even when most positions are correct, which is why sequence accuracy is usually lower than token accuracy on harder or longer-sequence tasks.

25. Consolidated Results

Students must complete the following table.

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	82.22	82.33	81.27	81.28	1974
LSTM	95.33	95.56	95.55	95.54	6006
GRU	94.67	95.07	94.93	94.97	4758
CNN-LSTM/GRU	100.00	100.00	100.00	100.00	168,163

A separate table must be provided for the sequence-to-sequence task.

Observed Output (Task 10 – Seq2Seq Results)

Metric	Value
Token Accuracy	100.00%
Sequence Accuracy	100.00%
Final Training Loss	0.0022
Final Validation Loss	0.0025

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

27. Discussion Questions

1. What is a recurrent neural network?
2. What information is represented by the hidden state?
3. Why is sequence order important?
4. Explain the difference between a feed-forward network and an RNN.
5. Explain Backpropagation Through Time.
6. Why can gradients vanish during BPTT?
7. Why can gradients explode during BPTT?
8. What are long-term dependencies?
9. What is the role of the LSTM cell state?
10. What are the forget, input and output gates in LSTM?
11. What are the reset and update gates in GRU?
12. Compare LSTM and GRU.
13. Why does GRU generally use a simpler gating mechanism than LSTM?
14. Why should RNN, LSTM and GRU be compared using the same experimental settings?
15. What does the confusion matrix reveal that accuracy alone does not?
16. Why should macro F1-score be reported for multi-class classification?
17. What is the significance of sequence length?
18. Why can a CNN be used as a feature extractor for video?
19. What type of information is extracted by the CNN?
20. What type of information is learned by the recurrent network?
21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?
22. Explain the encoder and decoder in sequence-to-sequence learning.
23. What is teacher forcing?
24. Differentiate token accuracy and sequence accuracy.
25. Why must the test set remain untouched during model selection?

28. Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.
2. Compare GRU and LSTM using the same number of recurrent units.
3. Add a second recurrent layer and investigate whether performance improves.
4. Compare a bidirectional LSTM with a unidirectional LSTM.
5. Change the sequence length and study the effect on computational cost.
6. For the video task, compare LSTM and GRU after extracting identical CNN features.
7. Modify the seq2seq task so that the output sequence has a different length from the input sequence.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>